

Version Control Systems

Created by <Academia de Código>

Version Control Systems (VCS)

management of changes in documents/code/information

No Version Control

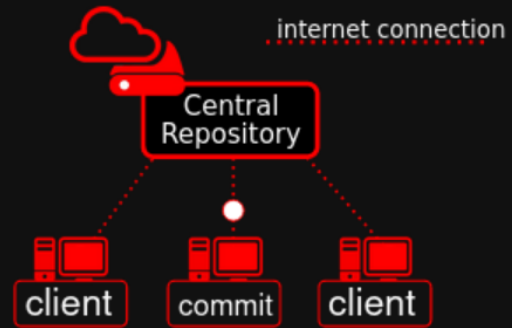


File Locks

File locks allow a developer to lock a file while working on it.

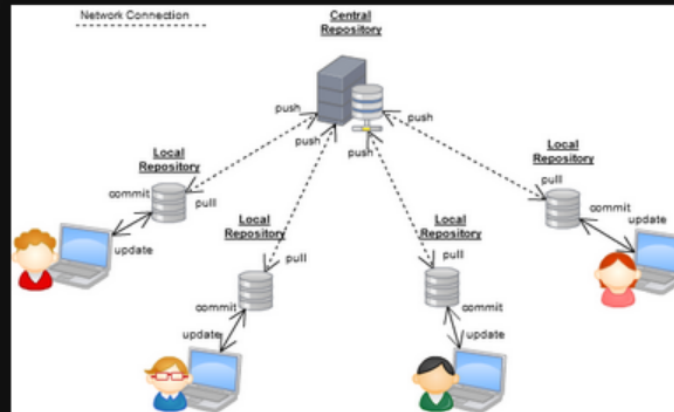
e.g. SCCS and RCS

Centralized Version Control Systems (CVCS)



e.g. CVS, SourceSafe, Perforce, SubVersion

Distributed Version Control Systems (DVCS)



e.g. Git, Mercurial, Bazaar, Bitkeeper



{ Hands On }

First-Time Git Setup

```
# check if git is installed
$ git version

# set user info and default editor
$ git config --global user.name "John Doe"
$ git config --global user.email johndoe@example.com
$ git config --global core.editor vim

# dump the git configuration and repo configuration if executed inside repo
$ git config --list

# getting help
$ git help config
```

Fake it 'til you make it

We'll learn what git is and how it works by using it.

Let's start off creating a fictitious project.

```
# create a new directory for your fake project, and move inside  
$ mkdir my-vegan-restaurant  
$ cd my-vegan-restaurant
```

Greetings, my name is git

It's time for git to know about your project.

Programmers would say we're initializing a local git repository inside your project root directory.

All your local repository data is stored inside a directory called `.git`. It is created as soon as you initialize the repository.

```
# initialize a local git repository
# (make sure your current directory is your project root directory!)
$ git init

# list all files (including hidden ones)
$ ls -a
```

Don't mess with the `.git` directory!

What are we doing here?

By now, you have successfully created a directory for your project and initialized a local git repository to manage its versions.

Git has nothing to keep track of, because there are no files inside your project directory.

Let's create your first file and check what that means to git.

```
# create your restaurant description
$ echo "We've got the best fresh vegetables all day, every day." > description.txt
# check your current repository status
$ git status
```

I might want to remember this

You are happy with the description you've come up with. You'd like to be able to come back to this one in case it changes for the worse.

This is where you make a new "version". Git calls it a *commit*.

A commit stores the state of your project (the content of your files) with an associated date and description.

Your commits are kept in a chronologically ordered list, and you are able to navigate between them (changing files' content on the fly).

It's okay to be picky

In most VCS, you would create a commit of your files content now.

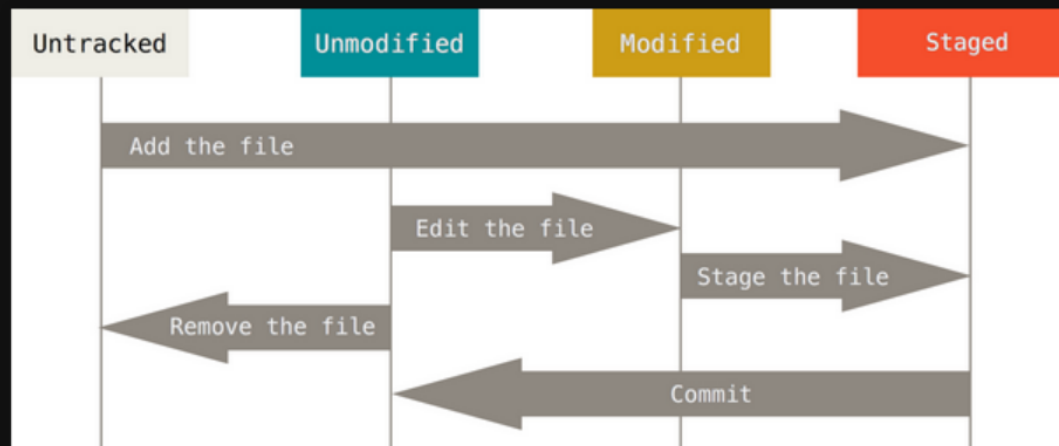
In git, you must add it to the staging area first. This *area* is where you place all the files you want to be *stored* in your next commit.

```
# add the file to the staging area
$ git add description.txt
# check your current repository status again
$ git status
```

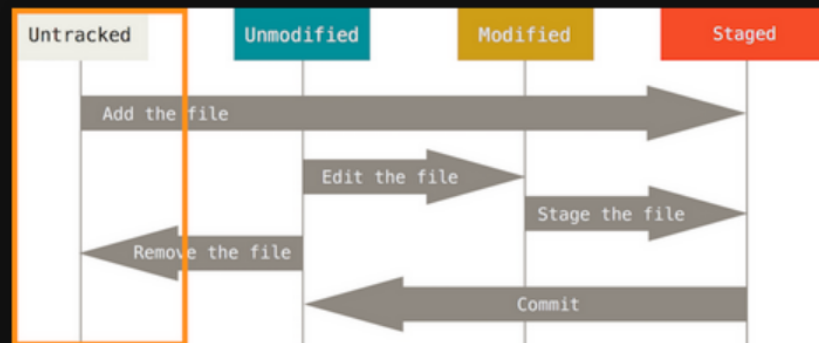
You may use unix metacharacters for pattern matching.

e.g: ., *, *.txt, etc..

File Status Lifecycle



File Lifecycle - Tracked/Untracked

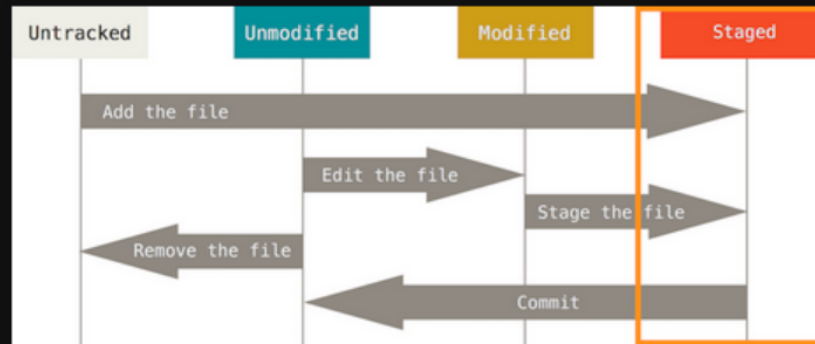


A file is said to be tracked if there is evidence of it in the last snapshot in the Local Repository.

If no story of it is available (files not yet committed for the first time), we say the file is untracked.

The git status report will list these files.

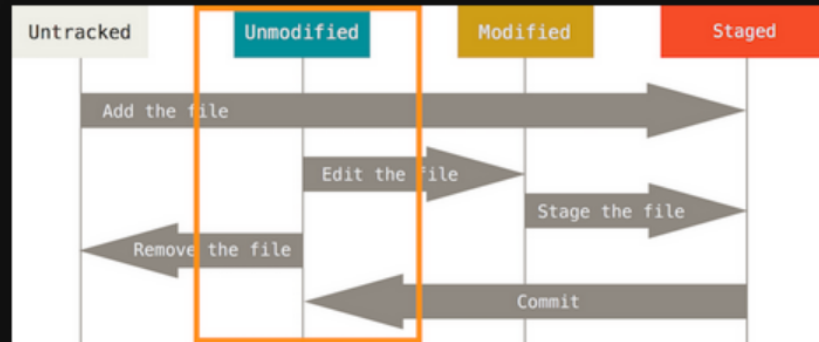
File Lifecycle - Add a file



When you add an untracked file, it goes directly to the staging area.

It will be added to the repository during the next commit

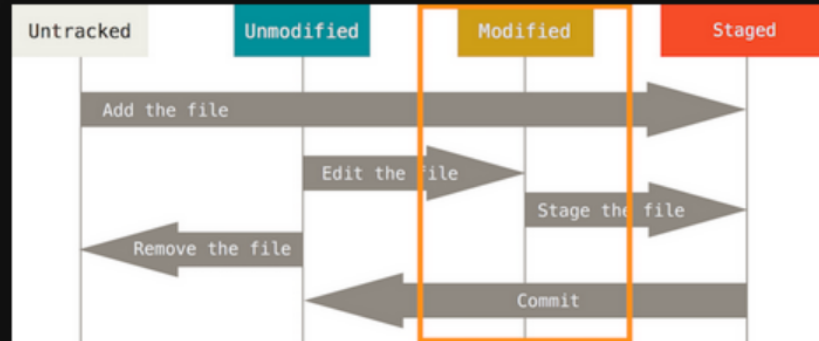
File Lifecycle - Commit a file



When you make a commit, staging area will flush and your files in the working directory will be in unmodified state.

This means the files in the working directory are the same as the snapshot in the local repository

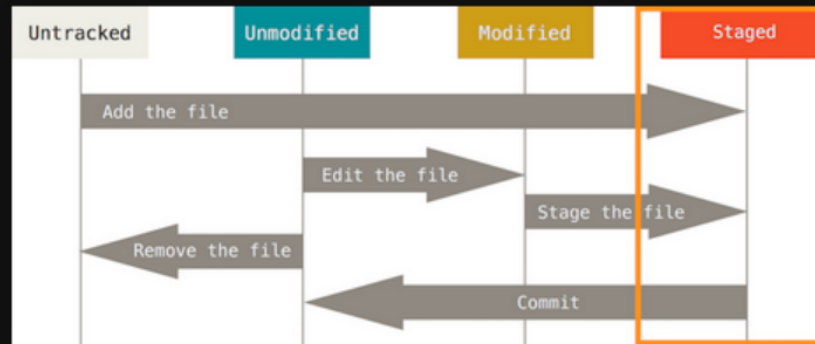
File Lifecycle - Modify a file



When you change a file in the working directory,
it changes to the modified state

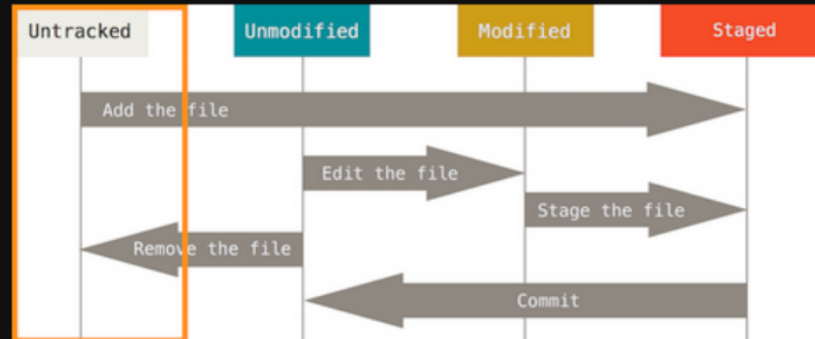
A modified file won't be automatically staged. You must manually do it (with the add command) if you want it to.

File Lifecycle - Stage a modified file



All the files in modified state can be added to the staging area
and will be ready to be part of the local repository during the next commit

File Lifecycle - Remove a file



A file can be removed from the unmodified state. This action will force the file to be untracked again

It won't be deleted from the working directory. You can always add it again, if needed

Ignoring files



We don't want git to track every file in our working directory

Do we always remember not to stage those files?

Does everyone in the team know what files not to stage?

No! So there must be a better way to do this!

.gitignore

.gitignore is a file that tells git
which files in the repository should be ignored

```
# create file .gitignore  
$ vim .gitignore
```

{ Hands On }

Ignore the rat problem in your restaurant

This is where it all started

After adding the file to the staging area, you are ready to make your first commit.

The commit needs to be accompanied with a brief description of what has changed.

It will be useful when navigating through hundreds of commits, or when you want to have an overview of what has been done in a series of commits.

```
# commit your staging area
$ git commit -m "Initial commit. Add restaurant description."
# I think you know what this means
$ git status
```

Remember this core concept: commit often, perfect later!

I'm getting the hang of this

Now that you've made your first commit, you are able to safely keep working on your project.

Let's add some menus to our restaurant!

```
# create a menus directory
$ mkdir menus
# create the fried cuttlefish menu
$ echo "Bean Burrito | Sweet Potato Fries | Beer -> \$5" > menus/bean-burrito.txt
# create a new menu you're still not sure about it's price
$ echo -n "Vegetable Curry | White Rice | Wine -> \$7" > menus/vegetable-curry.txt

$ git status
```

{ Exercise }

Make a new commit

It's time for you to create a new commit with both files and a nice description.

A piece of history

The two commits you've made are stored in your local repository and it's possible to see them in a chronologically ordered list.

A unique key is assigned to each commit that is used as an identifier. You'll use it whenever you want to reference a particular commit

```
# check your commit list  
$ git log
```

{ Exercise }

Raise your prices, and commit those changes

Change is inevitable

Just like you have changed your prices,
a software project keeps evolving over time.

As commits store the files content at a point in time, git also
provides a tool to compare file differences across versions.

```
# see the differences between two commits on a particular file
$ git diff COMMIT_ID menus/bean-burrito.txt
# see the differences between two commits (every file that has changed)
$ git diff COMMIT_ID
# see the differences between the last commit and the current project state (excluding
$ git diff
# see the differences between the last commit and the current staging area
$ git diff --staged
```

Get into the time machine!

Why would we store multiple versions of our files if we didn't want to check them in the future?

The clients didn't like the changes you've made to your prices, and you want to revert them as soon as possible.

Of course you could remember what the prices were in this situation, but imagine a restaurant with hundreds of menus that keep changing their price.

```
# revert a file to it's state in a particular commit
$ git checkout COMMIT_ID menus/bean-burrito.txt
# the file is automatically added to the staging area, you may commit the price revers:
$ git commit -m "revert burrito menu price"
# check your log for an overview
$ git log
```

Working with Remotes

Remote repositories are versions of your project that are hosted on the Internet or in a network somewhere.

{ Hands On }

Setup a Remote Repository

```
# Connect to the bootcamp server
ssh codecadet@codecadets.academiadecodigo.org

# create a new git shared repository
git init --bare myrepo.git

# logout from the bootcamp server
exit

# Connect the local and remote repositories
git remote add origin codecadet@codecadets.academiadecodigo.org:myrepo.git
```

{ Hands On }

Adding Remotes

```
# cloning a remote
$ git clone [url]

# adding a new remote repository, origin is a common shortname
$ git remote add [shortname] [url]

# listing all remotes with urls
$ git remote -v
```

{ Hands On }

Pulling and Pushing

```
# pull a copy of the master branch from the remote repository named origin,  
# and merges it with the current checked out local branch  
$ git pull origin master  
  
# merges current checked out local branch  
# with the remote branch the local branch was created from  
$ git pull  
  
# push data from local checked out branch  
# to master branch in the remote named origin  
$ git push origin master  
  
# push data from local checked out branch  
# to the remote branch the local branch was created from  
$ git push
```

Remember that you won't be able to push
if any changes are waiting for being pulled.